



Script-Controlled ACL – Restrict Record Access Based on Field Value

Project Report – ServiceNow Access Control List (ACL) Implementation

Team ID: SWTID-2026-8807

Team Leader: MOHAMED ABDUL KALAM M

Team Member 1: KABILNATH U

Team Member 2: IMTHIYAS AHAMED S

Team Member 3: KARTHICK S

1. Brainstorming and Ideation Phase

Organizations that manage student or institutional data through a platform such as ServiceNow face a common challenge: different departments or branches should only be able to view records that belong to them, while administrators need unrestricted access for governance and support. During the initial brainstorming stage, the project team explored how ServiceNow's built-in security framework could be used to enforce this kind of branch-level data segregation without duplicating tables or building a separate application for every branch.

Several approaches were discussed before finalizing the direction of the project:

- Creating separate tables for each branch (ECE, EEE, CSE) – rejected due to data duplication and maintenance overhead.
- Using simple role-based restriction on the table – rejected because it would block or allow entire tables rather than filtering by field value.
- Using a Script-Controlled Access Control List (ACL) that evaluates both the user's role and the value of the Branch field on each record – selected as the most flexible and scalable approach.

The core idea finalized was: build a single custom table, Institution Details (u_institution_details), that stores student records for all branches, and use scripted ACLs to dynamically permit or deny access to read, create, write, and delete operations based on the requesting user's role and the record's Branch value, while always granting full access to administrators.

2. Requirement Analysis Phase

2.1 Functional Requirements

- Only users belonging to the EEE branch (holding the custom role bb1) should be able to view EEE branch records.
- Administrators must retain full, unrestricted access to all records irrespective of branch.
- Users without the required role must not be able to view any institution records.
- Separate access levels must exist for Create, Read, Write, and Delete operations, each governed by its own custom role (bb1, bb2, bb3, bb4).
- The restriction must be enforced at the record level using the Branch field value, not merely at the table level.

2.2 Non-Functional Requirements

- Security: Access decisions must be enforced server-side through ACL scripts so they cannot be bypassed from the client (forms, lists, or Service Portal).
- Scalability: The solution should extend to additional branches or roles without redesigning the table structure.

- Auditability: Role assignments and ACL changes should be traceable through ServiceNow's standard administration screens.

2.3 Tools and Platform Requirements

- ServiceNow instance with administrator access.
- security_admin role (elevated) to create and modify Access Control Lists.
- A custom table to hold institution/student records.
- Test user accounts and custom roles to validate each access scenario.

3. Project Design Phase

3.1 Data Model Design

A custom table named Institution Details (u_institution_details) was designed to hold student records with the following fields:

Field Label	Type	Purpose
Student Roll Number	Auto Number	Unique identifier generated automatically for each record.
Student Name	Reference (User)	Links the record to the corresponding student user.
Faculty Name	Reference (User)	Links the record to the faculty responsible for the student.
Branch	Choice (ECE, EEE, CSE)	Determines which branch the record belongs to; used as the data condition in the ACL.
Email	String	Contact email of the student.
Phone Number	String	Contact number of the student.

Description	Multi-line String	Free-text notes; also used to confirm ACL execution during testing.
-------------	-------------------	---

3.2 Role Design

Four custom roles were designed, one per CRUD operation, so that access can be granted incrementally and independently:

Role	Operation Controlled
bb1	Read
bb2	Create
bb3	Write (Update)
bb4	Delete

3.3 Access Control Design

The Read ACL was designed as an Advanced (scripted) ACL combined with a data condition (Branch is EEE), so that access depends on both the user's role and the record's field value. The script logic is:

```
(function () { // Allow admin users full access  if
(gs.hasRole('admin')) {    return true;  } // Allow only EEE
branch users to see EEE records  if (gs.hasRole('bb1')) {    return
true;  } // Deny access for all others  return false; })();
```

The Create, Write, and Delete ACLs were designed as simple role-based ACLs (bb2, bb3, bb4 respectively) without a data condition, since these operations only needed to confirm that the user held the relevant role, while the Read ACL's data condition already restricts which specific records a role holder can see.

4. Project Planning Phase

The 3-hour lab was planned as a sequence of milestones, each building on the previous one so that the ACL behavior could be verified incrementally.

Milestone	Activity	Estimated Time
1	User and role creation (EEE User, roles bb1–bb4)	30 minutes
2	Custom table creation (u_institution_details) and field configuration	30 minutes
3	Sample record creation across ECE, EEE and CSE branches	20 minutes
4	Read ACL creation with script and data condition	30 minutes
5	Create, Write and Delete ACL creation	30 minutes
6	Verification through user impersonation for each role	30 minutes
7	Documentation of outcomes and screenshots	10 minutes

Dependencies were also planned: roles had to exist before they could be assigned to the EEE User or referenced inside the Requires role related list, and the table with its Branch field had to exist before any ACL referencing u_institution_details could be created.

5. Project Development Phase

5.1 User and Role Creation

1. Logged in to the ServiceNow instance with administrator access.
2. Navigated to User Administration → Users and created a new user (User ID: EEEUser, First name: EEE, Last name: User, Email: eeeuser@gmail.com).
3. Navigated to User Administration → Roles and created four custom roles: bb1, bb2, bb3, and bb4, each with a short description of its purpose.

4. Assigned all four roles to the EEE User record.

5.2 Table Creation

5. Searched System Definition → Tables and clicked New.
6. Created the table with Label: Institution Details, Name: u_institution_details, Extends: false.
7. Added the fields Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description as per the design.
8. Saved the table and created sample student records across the ECE, EEE, and CSE branches to support later testing.

5.3 Read ACL Development

9. Searched System Security → Access Control (ACL) and elevated to the security_admin role.
10. Created a new ACL with Type: record, Operation: read, Name: u_institution_details, Active: true, Advanced: true.
11. Added bb1 to the Requires role related list.
12. Added the data condition Branch is EEE.
13. Entered the scripted condition allowing admin users and bb1 role holders full read access, and denying all others.
14. Submitted the ACL record.

5.4 Create, Write and Delete ACL Development

15. Created a Create ACL (Type: record, Operation: create, Name: u_institution_details) with the role bb2 in Requires role, without any data condition.
16. Created a Write ACL (Type: record, Operation: write, Name: u_institution_details) with the role bb3 in Requires role.
17. Created a Delete ACL (Type: record, Operation: delete, Name: u_institution_details) with the role bb4 in Requires role.

18. Saved all ACL records under the same table so that access builds up progressively as a user is granted bb1 through bb4.

6. Project Testing Phase

Testing was carried out using ServiceNow's Impersonate user feature so that the behavior of each role could be verified from the perspective of an actual logged-in user, rather than only as an administrator.

6.1 Test Cases and Results

Test Case	Steps	Expected Result	Actual Result
Read access with bb1 role	Impersonate EEE User (with bb1) and open the Institution Details list.	Only records where Branch = EEE are visible.	Pass – only the two EEE branch records (Guna Sai, Prashanth Reddy) were displayed.
Read access without any role	Impersonate a user with no bb roles and open the Institution Details list.	No records are displayed.	Pass – list showed "No records to display".
Read access as Admin	Impersonate an Administrator and open the Institution Details list.	All records are visible regardless of branch.	Pass – all ECE, EEE and CSE records were visible.
Create access with bb1 & bb2	Impersonate user with bb1 and bb2 roles.	User can view EEE records and see the New button to create records.	Pass – New button was available and record creation succeeded.
Write access with bb1, bb2 & bb3	Impersonate user with bb1, bb2 and bb3 roles; edit the Description field.	User can view, create and edit EEE branch records.	Pass – Description field was editable and

			update saved successfully.
Delete access with bb1–bb4	Impersonate user with all four roles; select a record and choose Delete.	User can delete EEE branch records after confirmation.	Pass – selected record was removed after confirming the delete dialog.

All test cases passed, confirming that the scripted ACL correctly combines role checks with the Branch data condition, that administrators bypass the restriction entirely, and that users lacking the required roles are denied access at every operation level.

7. Project Documentation Phase

All configuration steps were documented with supporting screenshots at each stage, covering user creation, role creation, table and field configuration, and the creation of each of the four ACL records (Read, Create, Write, Delete). Each ACL record's Type, Operation, Name, Active flag, Advanced flag, data condition, script, and Requires role entries were captured so the configuration can be reproduced or audited later.

The verification results for each impersonated role were also documented with screenshots showing the exact list view returned to that user, providing evidence that the record-level restriction behaves as designed for every role combination (no role, bb1 only, bb1+bb2, bb1+bb2+bb3, and bb1+bb2+bb3+bb4) as well as for the administrator.

This documentation serves both as a lab record for learners and as a reference implementation that can be reused whenever a similar branch-level or department-level record restriction needs to be implemented on any other ServiceNow table.

8. Project Demonstration Phase

The completed solution was demonstrated end-to-end by walking through the full lifecycle of the ACL configuration and its effect on data visibility:

19. Demonstrated the Institution Details table containing mixed records for ECE, EEE and CSE branches, visible in full to the administrator.
20. Impersonated the EEE User and showed that only EEE branch records appeared in the list, proving the Read ACL's script and data condition were both being enforced.
21. Impersonated a user without the bb1 role and showed the empty list result, proving that the ACL denies access by default when no matching role is present.
22. Progressively added the bb2, bb3 and bb4 roles to the demonstration user and showed the New button, inline editing of the Description field, and the Delete action becoming available at each stage.
23. Confirmed that the Description field showed "ACL is Executed" after an edit, and that the selected record was successfully removed after the delete confirmation dialog, closing out the full CRUD demonstration.

The demonstration validated the outcome of the project: script-controlled READ, WRITE, CREATE and DELETE ACLs successfully enforce record-level security by evaluating user roles together with the record's Branch field value, while ensuring administrators retain unrestricted access across forms, lists and Service Portal interfaces.